E-Commerce Sales Analysis Using MySQL

Introduction

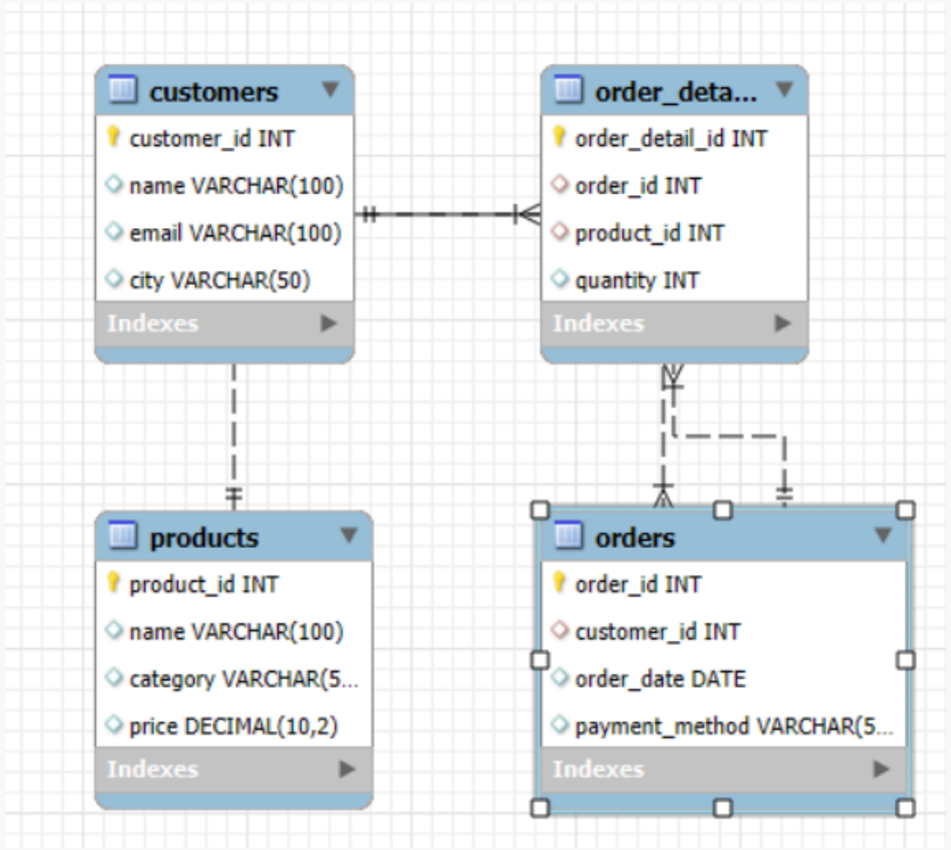
Hello, my name is Sailikith
In this project, I used SQL to analyze e-commerce sales data. I worked with a database called Ecommerce_Sales to answer questions related to sales trends, top-selling products, and customer behavior.
This project helped me apply real-world SQL concepts like joins, grouping, and subqueries to extract meaningful insights from the data.

ER Diagram of Ecommerce_Sales Database

This ER (Entity-Relationship) diagram shows the structure of the Ecommerce_Sales database.

It includes four main tables — Customers, Orders, Products, and Order_Details — and highlights the relationships between them.

These connections were essential for performing SQL joins and understanding how the data flows across the system.



Questions solved

Basic (5):

1. List all customers with their city and email.
2. Show all products with their category and price.
3. Find all orders made using 'Credit Card'.
4. Get details of products that cost more than ₹100.
5. Display all orders placed after a specific date.

Intermediate (5):

1. Total sales for each product.
2. Total number of orders per customer.
3. Top 3 products by total sales.
4. Average order value per payment method.
5. Orders with customer and product details.

Advanced (4):

1. Customer who spent the most in total.
2. Products never ordered.
3. View: order_id, customer_name, total order value.
4. Stored procedure: input month → output total monthly sales.

Basic level

1. List all customers with their city and email.

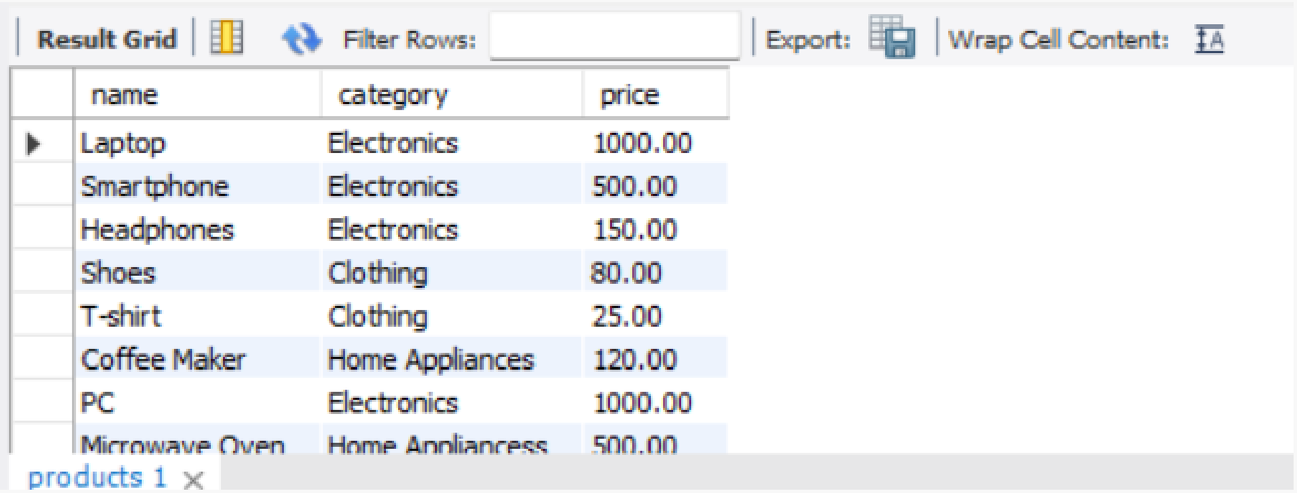
```
use orders;
SELECT
    name, city, email
FROM
    customers;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

name	city	email
John Doe	New York	john.doe@example.com
Jane Smith	Los Angeles	jane.smith@example.com
Michael Johnson	Chicago	michael.johnson@example.com
Sarah Lee	San Francisco	sarah.lee@example.com
David Brown	Miami	david.brown@example.com

2. Show all products with their category and price.

```sql
SELECT
    name, category, price
FROM
    products;
```



Result Grid | Filter Rows: | Export: | Wrap Cell Content:

name	category	price
Laptop	Electronics	1000.00
Smartphone	Electronics	500.00
Headphones	Electronics	150.00
Shoes	Clothing	80.00
T-shirt	Clothing	25.00
Coffee Maker	Home Appliances	120.00
PC	Electronics	1000.00
Microwave Oven	Home Appliances	500.00

products 1

3. Find all orders made using 'Credit Card'.

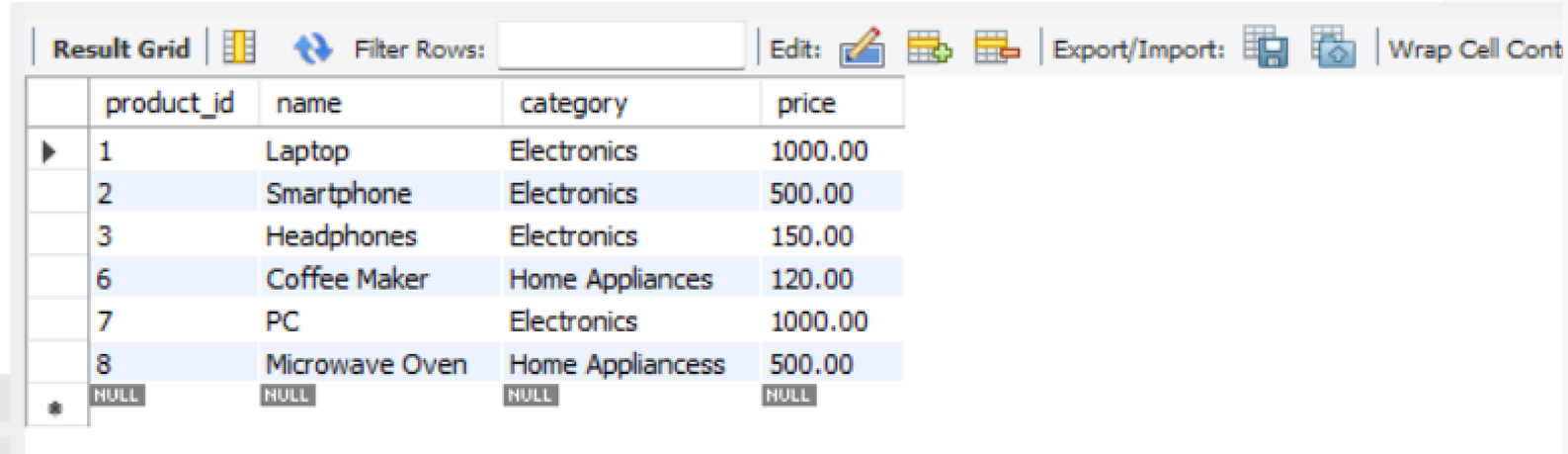
```sql
SELECT
    order_id
FROM
    orders
WHERE
    payment_method = 'Credit Card';
```

Result Grid | Filter Rows:

order_id
101
104
NULL

4. Get the details of products that cost more than ₹100.

```
select * from products where price>100;
```

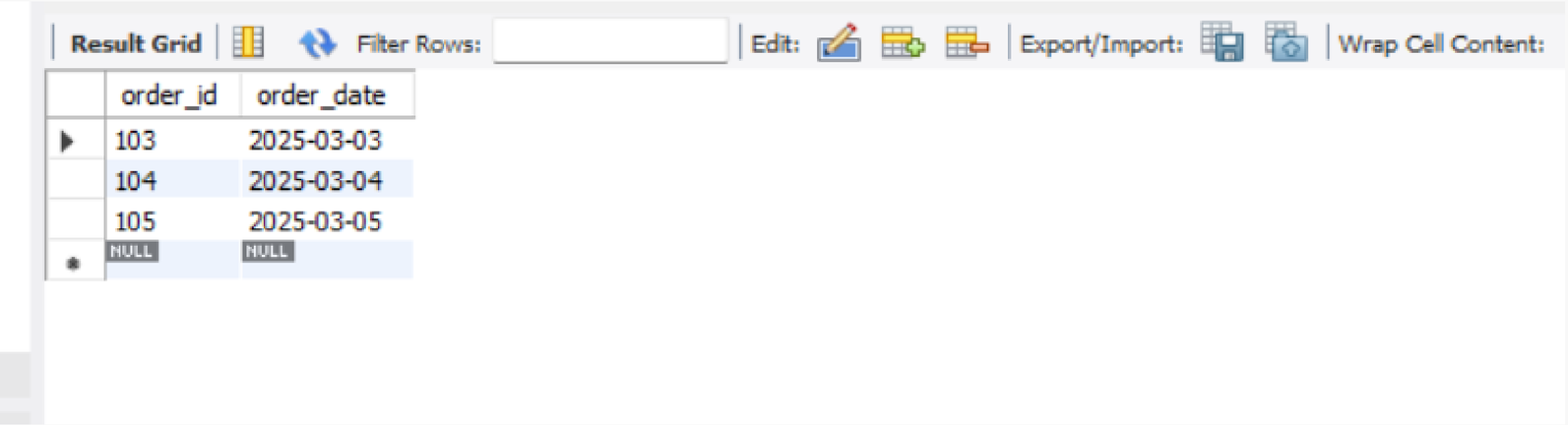


Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Cont

product_id	name	category	price
1	Laptop	Electronics	1000.00
2	Smartphone	Electronics	500.00
3	Headphones	Electronics	150.00
6	Coffee Maker	Home Appliances	120.00
7	PC	Electronics	1000.00
8	Microwave Oven	Home Appliancess	500.00
NULL	NULL	NULL	NULL

5. Display all orders placed after March 2, 2025.

```
select order_id,order_date from orders where order_date>'2025=03-02';
```

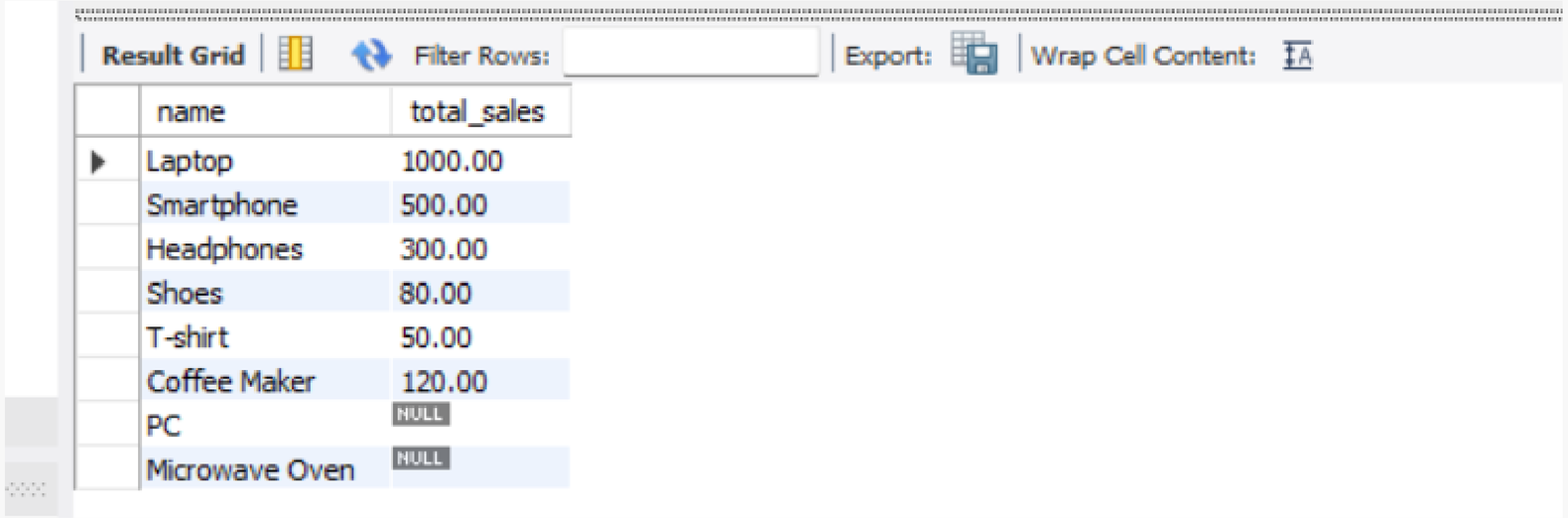


order_id	order_date
103	2025-03-03
104	2025-03-04
105	2025-03-05
NULL	NULL

Intermediate level

1. Find total sales for each product.

```sql
SELECT
    products.name,
    SUM(products.price * order_details.quantity) AS total_sales
FROM
    products
        LEFT JOIN
    order_details ON products.product_id = order_details.product_id
GROUP BY products.product_id;
```

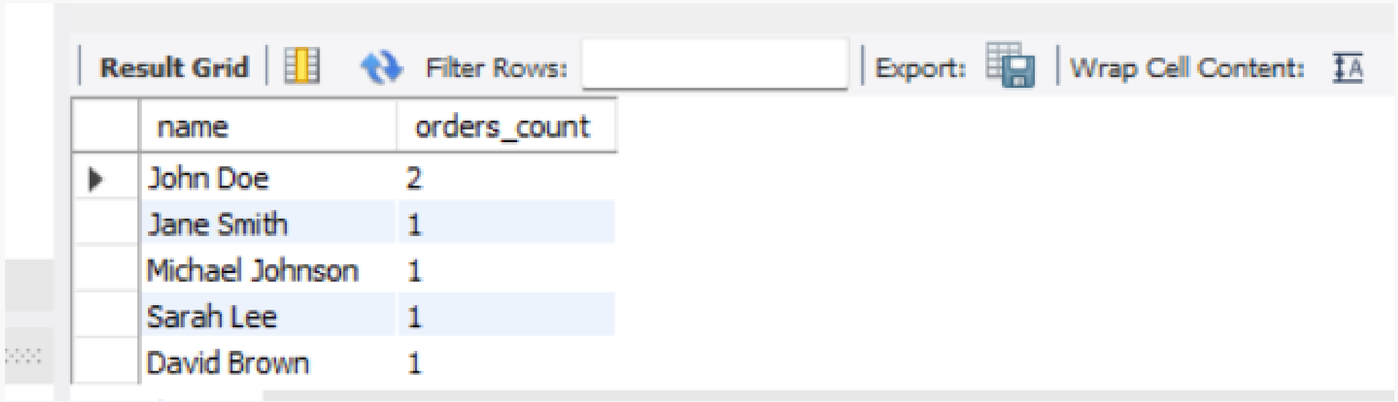


Result Grid | Filter Rows: | Export: | Wrap Cell Content:

name	total_sales
Laptop	1000.00
Smartphone	500.00
Headphones	300.00
Shoes	80.00
T-shirt	50.00
Coffee Maker	120.00
PC	NULL
Microwave Oven	NULL

2. List total number of orders placed by each customer.

```sql
SELECT
    customers.name, COUNT(order_details.order_id) as orders_count
FROM
    customers
        JOIN
    orders ON orders.customer_id = customers.customer_id
        JOIN
    order_details ON order_details.order_id = orders.order_id
GROUP BY customers.customer_id;
```

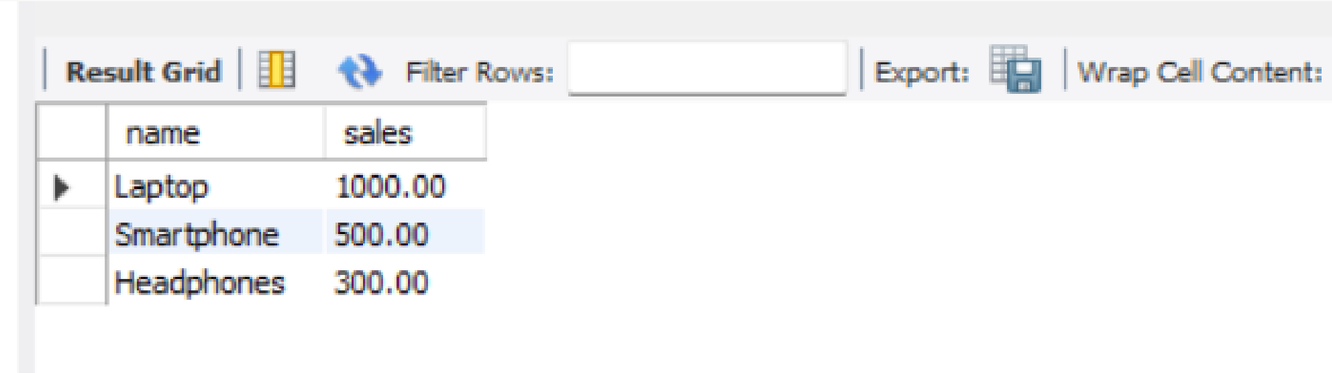


Result Grid | Filter Rows: | Export: | Wrap Cell Content:

name	orders_count
John Doe	2
Jane Smith	1
Michael Johnson	1
Sarah Lee	1
David Brown	1

3. Show the top 3 products by total sales amount.

```sql
SELECT
    products.name,
    SUM(products.price * order_details.quantity) AS sales
FROM
    products
        JOIN
    order_details ON products.product_id = order_details.product_id
GROUP BY products.name
ORDER BY sales DESC
LIMIT 3;
```



name	sales
Laptop	1000.00
Smartphone	500.00
Headphones	300.00

4. Get the average order value for each payment method.

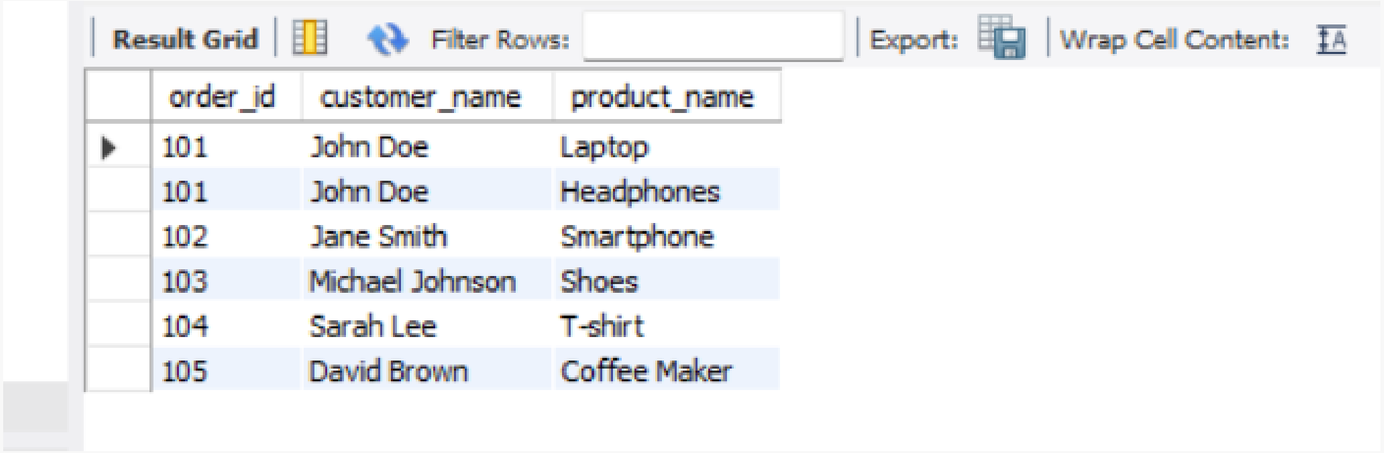
```sql
SELECT
    orders.payment_method,
    ROUND(AVG(products.price * order_details.quantity),
            2) as avg_order_value
FROM
    orders
        JOIN
    order_details ON orders.order_id = order_details.order_id
        JOIN
    products ON products.product_id = order_details.product_id
GROUP BY orders.payment_method;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

payment_method	avg_order_value
Credit Card	450.00
PayPal	310.00
Debit Card	80.00

5. Display all orders along with customer name and product names.

```
select order_details.order_id,customers.name as customer_name,products.name
as product_name from order_details join orders on
order_details.order_id=orders.order_id join
customers on customers.customer_id=orders.customer_id join
products on products.product_id=order_details.product_id;
```



order_id	customer_name	product_name
101	John Doe	Laptop
101	John Doe	Headphones
102	Jane Smith	Smartphone
103	Michael Johnson	Shoes
104	Sarah Lee	T-shirt
105	David Brown	Coffee Maker

Advance level

1. Find the customer who spent the most in total.

```sql
SELECT
    customers.name AS customer_name,
    SUM(products.price * order_details.quantity) AS Expenditure
FROM
    customers
        JOIN
    orders ON customers.customer_id = orders.customer_id
        JOIN
    order_details ON order_details.order_id = orders.order_id
        JOIN
    products ON products.product_id = order_details.product_id
GROUP BY customers.customer_id
ORDER BY Expenditure DESC
LIMIT 1;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

customer_name	Expenditure
John Doe	1300.00

2. List the products that were never ordered.

```sql
SELECT
    name, orders_count
FROM
    (SELECT
        products.name, COUNT(order_details.order_id) AS orders_count
    FROM
        products
    LEFT JOIN order_details ON products.product_id = order_details.product_id
    GROUP BY products.product_id) AS a
WHERE
    orders_count = 0;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

name	orders_count
PC	0
Microwave Oven	0

3. Create a view that shows order_id, customer_name, total order value.

```
create or replace view details as
select order_details.order_id,customers.name,(products.price*order_details.quantity)
as total_order_value from orders
join order_details on orders.order_id=order_details.order_id join
products on
products.product_id=order_details.product_id join
customers on customers.customer_id=orders.customer_id;

select * from details;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

order_id	name	total_order_value
101	John Doe	300.00
102	Jane Smith	500.00
103	Michael Johnson	80.00
104	Sarah Lee	50.00
105	David Brown	120.00

details 2

4. Write a stored procedure that takes a month (e.g., '03') as input and returns total sales in that month.

```
4   drop procedure if exists month_orders;
5
6   delimiter $$
7   create procedure month_orders(month_number int)
8   begin
9       select month(orders.order_date) as month,sum(products.price*order_details.quantity) as total_sales
10      from orders join order_details on orders.order_id=order_details.order_id
11      join products on order_details.product_id=products.product_id
12      where month(orders.order_date)=month_number
13      group by month;
14  end $$
15
16  delimiter ;
17
18  call month_orders(03)
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

month	total_sales
3	2050.00

Conclusion

This project demonstrates how SQL can be used for real-world business analysis through well-structured queries and relational database design. It provides insights that could help decision-makers in a retail business optimize their strategy.

Thank you